

Lua 5.5 Reference Manual

by Roberto Ierusalimschy, Luiz Henrique de Figueiredo, Waldemar Celes

Copyright © 2020–2026 Lua.org, PUC-Rio. Freely available under the terms of the Lua license.

contents · index · other versions

1 – Introduction

Lua is a powerful, efficient, lightweight, embeddable scripting language. It supports procedural programming, object-oriented programming, functional programming, data-driven programming, and data description.

Lua combines simple procedural syntax with powerful data description constructs based on associative arrays and extensible semantics. Lua is dynamically typed, runs by interpreting bytecode with a register-based virtual machine, and has automatic memory management with a generational garbage collection, making it ideal for configuration, scripting, and rapid prototyping.

Lua is implemented as a library, written in *clean C*, the common subset of standard C and C++. The Lua distribution includes a host program called `lua`, which uses the Lua library to offer a complete, standalone Lua interpreter, for interactive or batch use. Lua is intended to be used both as a powerful, lightweight, embeddable scripting language for any program that needs one, and as a powerful but lightweight and efficient stand-alone language.

As an extension language, Lua has no notion of a "main" program: it works *embedded* in a host client, called the *embedding program* or simply the *host*. (Frequently, this host is the stand-alone `lua` program.) The host program can invoke functions to execute a piece of Lua code, can write and read Lua variables, and can register C functions to be called by Lua code. Through the use of C functions, Lua can be augmented to cope with a wide range of different domains, thus creating customized programming languages sharing a syntactical framework.

Lua is free software, and is provided as usual with no guarantees, as stated in its license. The implementation described in this manual is available at Lua's official web site, `www.lua.org`.

Like any other reference manual, this document is dry in places. For a discussion of the decisions behind the design of Lua, see the technical papers available at Lua's web site. For a detailed introduction to programming in Lua, see Roberto's book, *Programming in Lua*.

2 – Basic Concepts

This section describes the basic concepts of the language.

2.1 – Values and Types

Lua is a dynamically typed language. This means that variables do not have types; only values do. There are no type definitions in the language. All values carry their own type.

All values in Lua are first-class values. This means that all values can be stored in variables, passed as arguments to other functions, and returned as results.

There are eight basic types in Lua: *nil*, *boolean*, *number*, *string*, *function*, *userdata*, *thread*, and *table*. The type *nil* has one single value, **nil**, whose main property is to be different from any other value; it often represents the absence of a useful value. The type *boolean* has two values, **false** and **true**. Both **nil** and **false** make a condition false; they are collectively called *false values*. Any other value makes a condition true. Despite its name, **false** is frequently used as an alternative to **nil**, with the key difference that **false** behaves like a regular value in a table, while a **nil** in a table represents an absent key.

The type *number* represents both integer numbers and real (floating-point) numbers, using two subtypes: *integer* and *float*. Standard Lua uses 64-bit integers and double-precision (64-bit) floats, but you can also compile Lua so that it uses 32-bit integers and/or single-precision (32-bit) floats. The option with 32 bits for both integers and floats is particularly attractive for small machines and embedded systems. (See macro `LUA_32BITS` in file `luaconf.h`.)

Unless stated otherwise, any overflow when manipulating integer values *wrap around*, according to the usual rules of two's complement arithmetic. (In other words, the actual result is the unique representable integer that is equal modulo 2^n to the mathematical result, where n is the number of bits of the integer type.)

Lua has explicit rules about when each subtype is used, but it also converts between them automatically as needed (see §3.4.3). Therefore, the programmer may choose to mostly ignore the difference between integers and floats or to assume complete control over the representation of each number.

The type *string* represents immutable sequences of bytes. Lua is 8-bit clean: strings can contain any 8-bit value, including embedded zeros (`'\0'`). Lua is also encoding-agnostic; it makes no assumptions about the contents of a string. The length of any string in Lua must fit in a Lua integer, and the string plus a small header must fit in `size_t`.

Lua can call (and manipulate) functions written in Lua and functions written in C (see §3.4.10). Both are represented by the type *function*.

The type *userdata* is provided to allow arbitrary C data to be stored in Lua variables. A userdata value represents a block of raw memory. There are two kinds of userdata: *full userdata*, which is an object with a block of memory managed by Lua, and *light userdata*, which is simply a C pointer value. Userdata has no predefined operations in Lua, except assignment and identity test. By using *metatables*, the programmer can define operations for full userdata values (see §2.4). Userdata values cannot be created or modified in Lua, only through the C API. This guarantees the integrity of data owned by the host program and C libraries.

The type *thread* represents independent threads of execution and it is used to implement coroutines (see §2.6). Lua threads are not related to operating-system threads. Lua supports coroutines on all systems, even those that do not support threads natively.

The type *table* implements associative arrays, that is, arrays that can have as indices not only numbers, but any Lua value except **nil** and NaN. (*Not a Number* is a special floating-point value used by the IEEE 754 standard to represent undefined numerical results, such as `0/0`.) Tables can be *heterogeneous*; that is, they can contain values of all types (except **nil**). Any key associated to the value **nil** is not considered part of the table. Conversely, any key that is not part of a table has an associated value **nil**.

Tables are the sole data-structuring mechanism in Lua; they can be used to represent ordinary arrays, lists, symbol tables, sets, records, graphs, trees, etc. To represent records, Lua uses the field name as an index. The language supports this representation by providing `a.name` as syntactic sugar for `a["name"]`. There are several convenient ways to create tables in Lua (see §3.4.9).

Like indices, the values of table fields can be of any type. In particular, because functions are first-class values, table fields can contain functions. Thus tables can also carry *methods* (see §3.4.11).

The indexing of tables follows the definition of raw equality in the language. The expressions `a[i]` and `a[j]` denote the same table element if and only if `i` and `j` are raw equal (that is, equal without metamethods). In particular, floats with integral values are equal to their respective integers (e.g., `1.0 == 1`). To avoid ambiguities, any float used as a key that is equal to an integer is converted to that integer. For instance, if you write `a[2.0] = true`, the actual key inserted into the table will be the integer `2`.

Tables, functions, threads, and (full) userdata values are *objects*: variables do not actually *contain* these values, only *references* to them. Assignment, parameter passing, and function returns always manipulate references to such values; these operations do not imply any kind of copy.

The library function `type` returns a string describing the type of a given value (see `type`).

2.2 – Scopes, Variables, and Environments

A variable name refers to a global or a local variable according to the declaration that is in context at that point of the code. (For the purposes of this discussion, a function's formal parameter is equivalent to a local variable.)

All chunks start with an implicit declaration `global *`, which declares all free names as global variables; this preambular declaration becomes void inside the scope of any other **global** declaration, as the following example illustrates:

```
X = 1       -- Ok, global by default
do
  global Y  -- voids the implicit initial declaration
  Y = 1     -- Ok, Y declared as global
  X = 1     -- ERROR, X not declared
end
X = 2       -- Ok, global by default again
```

So, outside any global declaration, Lua works as global-by-default. Inside any global declaration, Lua works without a default: All variables must be declared.

Lua is a lexically scoped language. The scope of a variable declaration begins at the first statement after the declaration and lasts until the last non-void statement of the innermost block that includes the declaration. (*Void statements* are labels and empty statements.)

A declaration shadows any declaration for the same name that is in context at the point of the declaration. Inside this shadow, any outer declaration for that name is void. See the next example:

```
global print, x
x = 10                -- global variable
do                    -- new block
  local x = x         -- new 'x', with value 10
  print(x)            --> 10
  x = x+1
  do                  -- another block
    local x = x+1     -- another 'x'
    print(x)          --> 12
  end
  print(x)            --> 11
end
print(x)              --> 10  (the global one)
```

Notice that, in a declaration like `local x = x`, the new `x` being declared is not in scope yet, and so the `x` on the right-hand side refers to the outside variable.

Because of the lexical scoping rules, local variables can be freely accessed by functions defined inside their scope. A local variable used by an inner function is called an *upvalue* (or *external local variable*, or simply *external variable*) inside the inner function.

Notice that each execution of a **local** statement defines new local variables. Consider the following example:

```
a = {}
local x = 20
for i = 1, 10 do
  local y = 0
  a[i] = function () y = y + 1; return x + y end
end
```

The loop creates ten closures (that is, ten instances of the anonymous function). Each of these closures uses a different `y` variable, while all of them share the same `x`.

As we will discuss further in §3.2 and §3.3.3, any reference to a global variable `var` is syntactically translated to `_ENV.var`. Moreover, every chunk is compiled in the scope of an external local variable named `_ENV` (see §3.3.2), so `_ENV` itself is never a free name in a chunk.

Despite the existence of this external `_ENV` variable and the translation of free names, `_ENV` is a regular name. In particular, you can define new variables and parameters with that name. (However, you should not define `_ENV` as a global variable, otherwise `_ENV.var` would translate to `_ENV._ENV.var` and so on, in an infinite loop.) Each reference to a global variable name uses the `_ENV` that is visible at that point in the program.

Any table used as the value of `_ENV` is called an *environment*.

Lua keeps a distinguished environment called the *global environment*. This value is kept at a special index in the C registry (see §4.3). In Lua, the global variable `_G` is initialized with this same value. (`_G` is never used internally, so changing its value will affect only your own code.)

When Lua loads a chunk, the default value for its `_ENV` variable is the global environment (see `load`). Therefore, by default, global variables in Lua code refer to entries in the global environment and, therefore, they act as conventional global variables. Moreover, all standard libraries are loaded in the global environment and some functions there operate on that environment. You can use `load` (or `loadfile`) to load a chunk with a different environment. (In C, you have to load the chunk and then change the value of its first upvalue; see `lua_setupvalue`.)

2.3 – Error Handling

Several operations in Lua can *raise* an error. An error interrupts the normal flow of the program, which can continue by *catching* the error.

Lua code can explicitly raise an error by calling the `error` function. (This function never returns.)

To catch errors in Lua, you can do a *protected call*, using `pcall` (or `xpcall`). The function `pcall` calls a given function in *protected mode*. Any error while running the function stops its execution, and control returns immediately to `pcall`, which returns a status code.

Because Lua is an embedded extension language, Lua code starts running by a call from C code in the host program. (When you use Lua standalone, the `lua` application is the host program.) Usually, this call is protected; so, when an otherwise unprotected error occurs during the compilation or execution of a Lua chunk, control returns to the host, which can take appropriate measures, such as printing an error message.

Whenever there is an error, an *error object* is propagated with information about the error. Lua itself only generates errors whose error object is a string, but programs can generate errors with any value as the error object, except **nil**. (Lua will change a **nil** as error object to a string message.) It is up to the Lua program or its host to handle such error objects. For historical reasons, an error object is often called an *error message*, even though it does not have to be a string.

When you use `xpcall` (or `lua_pcall`, in C) you can give a *message handler* to be called in case of errors. This function is called with the original error object and returns a new error object. It is called before the error unwinds the stack, so that it can gather more information about the error, for instance by inspecting the stack and creating a stack traceback. This message handler is still protected by the protected call; so, an error inside the message handler will call the message handler again. If this loop goes on for too long, Lua breaks it and returns an appropriate message. The message handler is called only for regular runtime errors. It is not called for memory-allocation errors nor for errors while running finalizers or other message handlers.

Lua also offers a system of *warnings* (see `warn`). Unlike errors, warnings do not interfere in any way with program execution. They typically only generate a message to the user, although this behavior can be adapted from C (see `lua_setwarnf`).

2.4 – Metatables and Metamethods

Every value in Lua can have a *metatable*. This *metatable* is an ordinary Lua table that defines the behavior of the original value under certain events. You can change several aspects of the behavior of a value by setting specific fields in its metatable. For instance, when a non-numeric value is the operand of an addition, Lua checks for a function in the field `__add` of the value's metatable. If it finds one, Lua calls this function to perform the addition.

The key for each event in a metatable is a string with the event name prefixed by two underscores; the corresponding value is called a *metavalue*. For most events, the metavalue must be a function, which is then called a *metamethod*. In the previous example, the key is the string "`__add`" and the metamethod is the function that performs the addition. Unless stated otherwise, a metamethod can in fact be any callable value, which is either a function or a value with a `__call` metamethod.

You can query the metatable of any value using the `getmetatable` function. Lua queries metamethods in metatables using a raw access (see `rawget`).

You can replace the metatable of tables using the `setmetatable` function. You cannot change the metatable of other types from Lua code, except by using the debug library (§6.11).

Tables and full userdata have individual metatables, although multiple tables and userdata can share their metatables. Values of all other types share one single metatable per type; that is, there is one single metatable for all numbers, one for all strings, etc. By default, a value has no metatable, but the string library sets a metatable for the string type (see §6.5).

A detailed list of operations controlled by metatables is given next. Each event is identified by its corresponding key. By convention, all metatable keys used by Lua are composed by two underscores followed by lowercase Latin letters.

- **`__add`**: the addition (+) operation. If any operand for an addition is not a number, Lua will try to call a metamethod. It starts by checking the first operand (even if it is a number); if that operand does not define a metamethod for `__add`, then Lua will check the second operand. If Lua can find a metamethod, it calls the metamethod with the two operands as arguments, and the result of the call (adjusted to one value) is the result of the operation. Otherwise, if no metamethod is found, Lua raises an error.
- **`__sub`**: the subtraction (`-`) operation. Behavior similar to the addition operation.
- **`__mul`**: the multiplication (`*`) operation. Behavior similar to the addition operation.
- **`__div`**: the division (`/`) operation. Behavior similar to the addition operation.
- **`__mod`**: the modulo (`%`) operation. Behavior similar to the addition operation.
- **`__pow`**: the exponentiation (`^`) operation. Behavior similar to the addition operation.
- **`__unm`**: the negation (unary `-`) operation. Behavior similar to the addition operation.
- **`__idiv`**: the floor division (`//`) operation. Behavior similar to the addition operation.
- **`__band`**: the bitwise AND (`&`) operation. Behavior similar to the addition operation, except that Lua will try a metamethod if any operand is neither an integer nor a float coercible to an integer (see §3.4.3).
- **`__bor`**: the bitwise OR (`|`) operation. Behavior similar to the bitwise AND operation.
- **`__bxor`**: the bitwise exclusive OR (binary ~) operation. Behavior similar to the bitwise AND operation.
- **`__bnot`**: the bitwise NOT (unary ~) operation. Behavior similar to the bitwise AND operation.
- **`__shl`**: the bitwise left shift (`<<`) operation. Behavior similar to the bitwise AND operation.
- **`__shr`**: the bitwise right shift (`>>`) operation. Behavior similar to the bitwise AND operation.
- **`__concat`**: the concatenation (`..`) operation. Behavior similar to the addition operation, except that Lua will try a metamethod if any operand is neither a string nor a number (which is always coercible to a string).
- **`__len`**: the length (`#`) operation. If the object is not a string, Lua will try its metamethod. If there is a metamethod, Lua calls it with the object as argument, and the result of the call (always adjusted to one value) is the result of the operation. If there is no metamethod but the object is a table, then Lua uses the table length operation (see §3.4.7). Otherwise, Lua raises an error.
- **`__eq`**: the equal (`==`) operation. Behavior similar to the addition operation, except that Lua will try a metamethod only when the values being compared are either both tables or both full

userdata and they are not primitively equal. The result of the call is always converted to a boolean.

- **`__lt`**: the less than (`<`) operation. Behavior similar to the addition operation, except that Lua will try a metamethod only when the values being compared are neither both numbers nor both strings. Moreover, the result of the call is always converted to a boolean.
- **`__le`**: the less equal (`<=`) operation. Behavior similar to the less than operation.
- **`__index`**: The indexing access operation `table[key]`. This event happens when `table` is not a table or when `key` is not present in `table`. The metavalue is looked up in the metatable of `table`.

  The metavalue for this event can be either a function, a table, or any value with an `__index` metavalue. If it is a function, it is called with `table` and `key` as arguments, and the result of the call (adjusted to one value) is the result of the operation. Otherwise, the final result is the result of indexing this metavalue with `key`. This indexing is regular, not raw, and therefore can trigger another `__index` metavalue.

- **`__newindex`**: The indexing assignment `table[key] = value`. Like the index event, this event happens when `table` is not a table or when `key` is not present in `table`. The metavalue is looked up in the metatable of `table`.

  Like with indexing, the metavalue for this event can be either a function, a table, or any value with an `__newindex` metavalue. If it is a function, it is called with `table`, `key`, and `value` as arguments. Otherwise, Lua repeats the indexing assignment over this metavalue with the same key and value. This assignment is regular, not raw, and therefore can trigger another `__newindex` metavalue.

  Whenever a `__newindex` metavalue is invoked, Lua does not perform the primitive assignment. If needed, the metamethod itself can call `rawset` to do the assignment.

- **`__call`**: The call operation `func(args)`. This event happens when Lua tries to call a non-function value (that is, `func` is not a function). The metamethod is looked up in `func`. If present, the metamethod is called with `func` as its first argument, followed by the arguments of the original call (`args`). All results of the call are the results of the operation. This is the only metamethod that allows multiple results.

In addition to the previous list, the interpreter also respects the following keys in metatables: `__gc` (see §2.5.3), `__close` (see §3.3.8), `__mode` (see §2.5.4), and `__name`. (The entry `__name`, when it contains a string, may be used by `tostring` and in error messages.)

For the unary operators (negation, length, and bitwise NOT), the metamethod is computed and called with a dummy second operand, equal to the first one. This extra operand is only to simplify Lua's internals (by making these operators behave like a binary operation) and may be removed in future versions. For most uses this extra operand is irrelevant.

Because metatables are regular tables, they can contain arbitrary fields, not only the event names defined above. Some functions in the standard library (e.g., `tostring`) use other fields in metatables for their own purposes.

It is a good practice to add all needed metamethods to a table before setting it as a metatable of some object. In particular, the `__gc` metamethod works only when this order is followed (see §2.5.3). It is also a good practice to set the metatable of an object right after its creation.

2.5 – Garbage Collection

Lua performs automatic memory management. This means that you do not have to worry about allocating memory for new objects or freeing it when the objects are no longer needed. Lua manages memory automatically by running a *garbage collector* to collect all *dead* objects. All memory used by Lua is subject to automatic management: strings, tables, userdata, functions, threads, internal structures, etc.

An object is considered *dead* as soon as the collector can be sure the object will not be accessed again in the normal execution of the program. ("Normal execution" here excludes finalizers, which resurrect dead objects (see §2.5.3), and it excludes also some operations using the debug library.) Note that the time when the collector can be sure that an object is dead may not coincide with the programmer's expectations. The only guarantees are that Lua will not collect an object that may still be accessed in the normal execution of the program, and it will eventually collect an object that is inaccessible from Lua. (Here, *inaccessible from Lua* means that neither a variable nor another live object refer to the object.) Because Lua has no knowledge about C code, it never collects objects accessible through the registry (see §4.3), which includes the global environment (see §2.2) and the main thread.

The garbage collector (GC) in Lua can work in two modes: incremental and generational.

The default GC mode with the default parameters are adequate for most uses. However, programs that waste a large proportion of their time allocating and freeing memory can benefit from other settings. Keep in mind that the GC behavior is non-portable both across platforms and across different Lua releases; therefore, optimal settings are also non-portable.

You can change the GC mode and parameters by calling `lua_gc` in C or `collectgarbage` in Lua. You can also use these functions to control the collector directly, for instance to stop or restart it.

2.5.1 – Incremental Garbage Collection

In incremental mode, each GC cycle performs a mark-and-sweep collection in small steps interleaved with the program's execution. In this mode, the collector uses three numbers to control its garbage-collection cycles: the *garbage-collector pause*, the *garbage-collector step multiplier*, and the *garbage-collector step size*.

The garbage-collector pause controls how long the collector waits before starting a new cycle. The collector starts a new cycle when the number of bytes hits *n%* of the total after the previous collection. Larger values make the collector less aggressive. Values equal to or less than 100 mean the collector will not wait to start a new cycle. A value of 200 means that the collector waits for the total number of bytes to double before starting a new cycle.

The garbage-collector step size controls the size of each incremental step, specifically how many bytes the interpreter allocates before performing a step: A value of *n* means the interpreter will allocate approximately *n* bytes between steps.

The garbage-collector step multiplier controls how much work each incremental step does. A value of *n* means the interpreter will execute *n% units of work* for each word allocated. A unit of work corresponds roughly to traversing one slot or sweeping one object. Larger values make the collector more aggressive. Beware that values too small can make the collector too slow to ever finish a cycle. As a special case, a zero value means unlimited work, effectively producing a non-incremental, stop-the-world collector.

2.5.2 – Generational Garbage Collection

In generational mode, the collector does frequent *minor* collections, which traverses only objects recently created. If after a minor collection the number of bytes is above a limit, the collector shifts to a *major* collection, which traverses all objects. The collector will then stay doing major collections until it detects that the program is generating enough garbage to justify going back to minor collections.

The generational mode uses three parameters: the *minor multiplier*, the *minor-major multiplier*, and the *major-minor multiplier*.

The minor multiplier controls the frequency of minor collections. For a minor multiplier *x*, a new minor collection will be done when the number of bytes grows *x%* larger than the number in use just after the last major collection. For instance, for a multiplier of 20, the collector will do a minor collection when the number of bytes gets 20% larger than the total after the last major collection.

The minor-major multiplier controls the shift to major collections. For a multiplier *x*, the collector will shift to a major collection when the number of bytes from old objects grows *x%* larger than the total after the previous major collection. For instance, for a multiplier of 100, the collector will do a major collection when the number of old bytes gets larger than twice the total after the previous major collection. As a special case, a value of 0 stops the collector from doing major collections.

The major-minor multiplier controls the shift back to minor collections. For a multiplier *x*, the collector will shift back to minor collections after a major collection collects at least *x%* of the bytes allocated during the last cycle. In particular, for a multiplier of 0, the collector will immediately shift back to minor collections after doing one major collection.

2.5.3 – Garbage-Collection Metamethods

You can set garbage-collector metamethods for tables and, using the C API, for full userdata (see §2.4). These metamethods, called *finalizers*, are called when the garbage collector detects that the corresponding table or userdata is dead. Finalizers allow you to coordinate Lua's garbage collection with external resource management such as closing files, network or database connections, or freeing your own memory.

For an object (table or userdata) to be finalized when collected, you must *mark* it for finalization. You mark an object for finalization when you set its metatable and the metatable has a `__gc` metamethod. Note that if you set a metatable without a `__gc` field and later create that field in the metatable, the object will not be marked for finalization.

When a marked object becomes dead, it is not collected immediately by the garbage collector. Instead, Lua puts it in a list. After the collection, Lua goes through that list. For each object in the list, it checks the object's `__gc` metamethod: If it is present, Lua calls it with the object as its single argument.

At the end of each garbage-collection cycle, the finalizers are called in the reverse order that the objects were marked for finalization, among those collected in that cycle; that is, the first finalizer to be called is the one associated with the object marked last in the program. The execution of each finalizer may occur at any point during the execution of the regular code.

Because the object being collected must still be used by the finalizer, that object (and other objects accessible only through it) must be *resurrected* by Lua. Usually, this resurrection is transient, and the object memory is freed in the next garbage-collection cycle. However, if the finalizer stores the object in some global place (e.g., a global variable), then the resurrection is permanent. Moreover, if the finalizer marks a finalizing object for finalization again, its finalizer will be called again in the next

cycle where the object is dead. In any case, the object memory is freed only in a GC cycle where the object is dead and not marked for finalization.

When you close a state (see `lua_close`), Lua calls the finalizers of all objects marked for finalization, following the reverse order that they were marked. If any finalizer marks objects for collection during that phase, these marks have no effect.

Finalizers cannot yield nor run the garbage collector. Because they can run in unpredictable times, it is good practice to restrict each finalizer to the minimum necessary to properly release its associated resource.

Any error while running a finalizer generates a warning; the error is not propagated.

2.5.4 – Weak Tables

A *weak table* is a table whose elements are *weak references*. A weak reference is ignored by the garbage collector. In other words, if the only references to an object are weak references, then the garbage collector will collect that object.

A weak table can have weak keys, weak values, or both. A table with weak values allows the collection of its values, but prevents the collection of its keys. A table with both weak keys and weak values allows the collection of both keys and values. In any case, if either the key or the value is collected, the whole pair is removed from the table. The weakness of a table is controlled by the `__mode` field of its metatable. This metavalue, if present, must be one of the following strings: "k", for a table with weak keys; "v", for a table with weak values; or "kv", for a table with both weak keys and values.

A table with weak keys and strong values is also called an *ephemeron table*. In an ephemeron table, a value is considered reachable only if its key is reachable. In particular, if the only reference to a key comes through its value, the pair is removed.

Any change in the weakness of a table may take effect only at the next collect cycle. In particular, if you change the weakness to a stronger mode, Lua may still collect some items from that table before the change takes effect.

Only objects that have an explicit construction are removed from weak tables. Values, such as numbers and light C functions, are not subject to garbage collection, and therefore are not removed from weak tables (unless their associated values are collected). Although strings are subject to garbage collection, they do not have an explicit construction and their equality is by value; they behave more like values than like objects. Therefore, they are not removed from weak tables.

Resurrected objects (that is, objects being finalized and objects accessible only through objects being finalized) have a special behavior in weak tables. They are removed from weak values before running their finalizers, but are removed from weak keys only in the next collection after running their finalizers, when such objects are actually freed. This behavior allows the finalizer to access properties associated with the object through weak tables.

If a weak table is among the resurrected objects in a collection cycle, it may not be properly cleared until the next cycle.

2.6 – Coroutines

Lua supports coroutines, also called *collaborative multithreading*. A coroutine in Lua represents an independent thread of execution. Unlike threads in multithread systems, however, a coroutine only suspends its execution by explicitly calling a yield function.

You create a coroutine by calling `coroutine.create`. Its sole argument is a function that is the main function of the coroutine. The `create` function only creates a new coroutine and returns a handle to it (an object of type *thread*); it does not start the coroutine.

You execute a coroutine by calling `coroutine.resume`. When you first call `coroutine.resume`, passing as its first argument a thread returned by `coroutine.create`, the coroutine starts its execution by calling its main function. Extra arguments passed to `coroutine.resume` are passed as arguments to that function. After the coroutine starts running, it runs until it terminates or *yields*.

A coroutine can terminate its execution in two ways: normally, when its main function returns (explicitly or implicitly, after the last instruction); and abnormally, if there is an unprotected error. In case of normal termination, `coroutine.resume` returns **true**, plus any values returned by the coroutine main function. In case of errors, `coroutine.resume` returns **false** plus the error object. In this case, the coroutine does not unwind its stack, so that it is possible to inspect it after the error with the debug API.

A coroutine yields by calling `coroutine.yield`. When a coroutine yields, the corresponding `coroutine.resume` returns immediately, even if the yield happens inside nested function calls (that is, not in the main function, but in a function directly or indirectly called by the main function). In the case of a yield, `coroutine.resume` also returns **true**, plus any values passed to `coroutine.yield`. The next time you resume the same coroutine, it continues its execution from the point where it yielded, with the call to `coroutine.yield` returning any extra arguments passed to `coroutine.resume`.

Like `coroutine.create`, the `coroutine.wrap` function also creates a coroutine, but instead of returning the coroutine itself, it returns a function that, when called, resumes the coroutine. Any arguments passed to this function go as extra arguments to `coroutine.resume`. `coroutine.wrap` returns all the values returned by `coroutine.resume`, except the first one (the boolean error code). Unlike `coroutine.resume`, the function created by `coroutine.wrap` propagates any error to the caller. In this case, the function also closes the coroutine (see `coroutine.close`).

As an example of how coroutines work, consider the following code:

```
function foo (a)
  print("foo", a)
  return coroutine.yield(2*a)
end

co = coroutine.create(function (a,b)
      print("co-body", a, b)
      local r = foo(a+1)
      print("co-body", r)
      local r, s = coroutine.yield(a+b, a-b)
      print("co-body", r, s)
      return b, "end"
end)

print("main", coroutine.resume(co, 1, 10))
print("main", coroutine.resume(co, "r"))
```

```
print("main", coroutine.resume(co, "x", "y"))
print("main", coroutine.resume(co, "x", "y"))
```

When you run it, it produces the following output:

```
co-body 1       10
foo     2
main    true    4
co-body r
main    true    11      -9
co-body x       y
main    true    10      end
main    false   cannot resume dead coroutine
```

You can also create and manipulate coroutines through the C API: see functions `lua_newthread`, `lua_resume`, and `lua_yield`.

3 – The Language

This section describes the lexis, the syntax, and the semantics of Lua. In other words, this section describes which tokens are valid, how they can be combined, and what their combinations mean.

Language constructs will be explained using the usual extended BNF notation, in which {*a*} means 0 or more *a*'s, and [*a*] means an optional *a*. Non-terminals are shown like non-terminal, keywords are shown like **kword**, and other terminal symbols are shown like '**=**'. The complete syntax of Lua can be found in §9 at the end of this manual.

3.1 – Lexical Conventions

Lua is a free-form language. It ignores spaces and comments between lexical elements (tokens), except as delimiters between two tokens. In source code, Lua recognizes as spaces the standard ASCII whitespace characters space, form feed, newline, carriage return, horizontal tab, and vertical tab.

Names (also called *identifiers*) in Lua can be any string of Latin letters, Arabic-Indic digits, and underscores, not beginning with a digit and not being a reserved word. Identifiers are used to name variables, table fields, and labels.

The following *keywords* are reserved and cannot be used as names:

```
and       break     do        else      elseif    end
false     for       function  global    goto      if
in        local     nil       not       or        repeat
return    then      true      until     while
```

Lua is a case-sensitive language: `and` is a reserved word, but `And` and `AND` are two different, valid names. As a convention, programs should avoid creating names that start with an underscore followed by one or more uppercase letters (such as `_VERSION`).

The following strings denote other tokens:

```
+     -     *     /     %     ^     #
&     ~     |     <<    >>    //
==    ~=    <=    >=    <     >     =
(     )     {     }     [     ]     ::
;     :     ,     .     ..    ...
```

A *short literal string* can be delimited by matching single or double quotes, and can contain the following C-like escape sequences: '\a' (bell), '\b' (backspace), '\f' (form feed), '\n' (newline), '\r' (carriage return), '\t' (horizontal tab), '\v' (vertical tab), '\\' (backslash), '\"' (quotation mark [double quote]), and '\'' (apostrophe [single quote]). A backslash followed by a line break results in a newline in the string. The escape sequence '\z' skips the following span of whitespace characters, including line breaks; it is particularly useful to break and indent a long literal string into multiple lines without adding the newlines and spaces into the string contents. A short literal string cannot contain unescaped line breaks nor escapes not forming a valid escape sequence.

We can specify any byte in a short literal string, including embedded zeros, by its numeric value. This can be done with the escape sequence \x*XX*, where *XX* is a sequence of exactly two hexadecimal digits, or with the escape sequence *ddd*, where *ddd* is a sequence of up to three decimal digits. (Note that if a decimal escape sequence is to be followed by a digit, it must be expressed using exactly three digits.)

The UTF-8 encoding of a Unicode character can be inserted in a literal string with the escape sequence \u{*XXX*} (with mandatory enclosing braces), where *XXX* is a sequence of one or more hexadecimal digits representing the character code point. This code point can be any value less than 2^{31}. (Lua uses the original UTF-8 specification here, which is not restricted to valid Unicode code points.)

Literal strings can also be defined using a long format enclosed by *long brackets*. We define an *opening long bracket of level n* as an opening square bracket followed by *n* equal signs followed by another opening square bracket. So, an opening long bracket of level 0 is written as [[, an opening long bracket of level 1 is written as [=[, and so on. A *closing long bracket* is defined similarly; for instance, a closing long bracket of level 4 is written as]====]. A *long literal* starts with an opening long bracket of any level and ends at the first closing long bracket of the same level. It can contain any text except a closing bracket of the same level. Literals in this bracketed form can run for several lines, do not interpret any escape sequences, and ignore long brackets of any other level. Any kind of end-of-line sequence (carriage return, newline, carriage return followed by newline, or newline followed by carriage return) is converted to a simple newline. When the opening long bracket is immediately followed by a newline, the newline is not included in the string.

As an example, in a system using ASCII (in which 'a' is coded as 97, newline is coded as 10, and '1' is coded as 49), the five literal strings below denote the same string:

```
a = 'alo\n123"'
a = "alo\n123\""
a = '\97lo\10\04923"'
a = [[alo
123"]]
a = [==[
alo
123"]==]
```

Any byte in a literal string not explicitly affected by the previous rules represents itself. However, Lua opens files for parsing in text mode, and the system's file functions may have problems with some

control characters. So, it is safer to represent binary data as a quoted literal with explicit escape sequences for the non-text characters.

A *numeric constant* (or *numeral*) can be written with an optional fractional part and an optional decimal exponent, marked by a letter 'e' or 'E'. Lua also accepts hexadecimal constants, which start with `0x` or `0X`. Hexadecimal constants also accept an optional fractional part plus an optional binary exponent, marked by a letter 'p' or 'P' and written in decimal. (For instance, `0x1.fp10` denotes 1984, which is *0x1f / 16* multiplied by 2^{10}.)

A numeric constant with a radix point or an exponent denotes a float; otherwise, if its value fits in an integer or it is a hexadecimal constant, it denotes an integer; otherwise (that is, a decimal integer numeral that overflows), it denotes a float. Hexadecimal numerals with neither a radix point nor an exponent always denote an integer value; if the value overflows, it *wraps around* to fit into a valid integer.

Examples of valid integer constants are

```
3   345   0xff   0xBEBADA
```

Examples of valid float constants are

```
3.0     3.1416     314.16e-2     0.31416E1     34e1
0x0.1E  0xA23p-4   0X1.921FB54442D18P+1
```

A *comment* starts with a double hyphen (`--`) anywhere outside a string. If the text immediately after `--` is not an opening long bracket, the comment is a *short comment*, which runs until the end of the line. Otherwise, it is a *long comment*, which runs until the corresponding closing long bracket.

3.2 – Variables

Variables are places that store values. There are three kinds of variables in Lua: global variables, local variables, and table fields.

A single name can denote a global variable or a local variable (or a function's formal parameter, which is a particular kind of local variable) (see §2.2):

```
var ::= Name
```

Name denotes identifiers (see §3.1).

Because variables are *lexically scoped*, local variables can be freely accessed by functions defined inside their scope (see §2.2).

Before the first assignment to a variable, its value is **nil**.

Square brackets are used to index a table:

```
var ::= prefixexp ‘[’ exp ‘]’
```

The meaning of accesses to table fields can be changed via metatables (see §2.4).

The syntax `var.Name` is just syntactic sugar for `var["Name"]`:

```
var ::= prefixexp ‘.’ Name
```

An access to a global variable `x` is equivalent to `_ENV.x`.

3.3 – Statements

Lua supports an almost conventional set of statements, similar to those in other conventional languages. This set includes blocks, assignments, control structures, function calls, and variable declarations.

3.3.1 – Blocks

A block is a list of statements, which are executed sequentially:

```
block ::= {stat}
```

Lua has *empty statements* that allow you to separate statements with semicolons, start a block with a semicolon or write two semicolons in sequence:

```
stat ::= ';'
```

Both function calls and assignments can start with an open parenthesis. This possibility leads to an ambiguity in Lua's grammar. Consider the following fragment:

```
a = b + c
(print or io.write)('done')
```

The grammar could see this fragment in two ways:

```
a = b + c(print or io.write)('done')

a = b + c; (print or io.write)('done')
```

The current parser always sees such constructions in the first way, interpreting the open parenthesis as the start of the arguments to a call. To avoid this ambiguity, it is a good practice to always precede with a semicolon statements that start with a parenthesis:

```
;(print or io.write)('done')
```

A block can be explicitly delimited to produce a single statement:

```
stat ::= do block end
```

Explicit blocks are useful to control the scope of variable declarations. Explicit blocks are also sometimes used to add a **return** statement in the middle of another block (see §3.3.4).

3.3.2 – Chunks

The unit of compilation of Lua is called a *chunk*. Syntactically, a chunk is simply a block:

```
chunk ::= block
```

Lua handles a chunk as the body of an anonymous function with a variable number of arguments (see §3.4.11). As such, chunks can define local variables, receive arguments, and return values. Moreover, such anonymous function is compiled as in the scope of an external local variable called

`_ENV` (see §2.2). The resulting function always has `_ENV` as its only external variable, even if it does not use that variable.

A chunk can be stored in a file or in a string inside the host program. To execute a chunk, Lua first *loads* it, precompiling the chunk's code into instructions for a virtual machine, and then Lua executes the compiled code with an interpreter for the virtual machine.

Chunks can also be precompiled into binary form; see the program `luac` and the function `string.dump` for details. Programs in source and compiled forms are interchangeable; Lua automatically detects the file type and acts accordingly (see `load`). Be aware that, unlike source code, maliciously crafted binary chunks can crash the interpreter.

3.3.3 – Assignment

Lua allows multiple assignments. Therefore, the syntax for assignment defines a list of variables on the left side and a list of expressions on the right side. The elements in both lists are separated by commas:

```
stat ::= varlist ‘=’ explist
varlist ::= var {‘,’ var}
explist ::= exp {‘,’ exp}
```

Expressions are discussed in §3.4.

Before the assignment, the list of values is *adjusted* to the length of the list of variables (see §3.4.12).

If a variable is both assigned and read inside a multiple assignment, Lua ensures that all reads get the value of the variable before the assignment. Thus the code

```
i = 3
i, a[i] = i+1, 20
```

sets `a[3]` to 20, without affecting `a[4]` because the `i` in `a[i]` is evaluated (to 3) before it is assigned 4. Similarly, the line

```
x, y = y, x
```

exchanges the values of `x` and `y`, and

```
x, y, z = y, z, x
```

cyclically permutes the values of `x`, `y`, and `z`.

Note that this guarantee covers only accesses syntactically inside the assignment statement. If a function or a metamethod called during the assignment changes the value of a variable, Lua gives no guarantees about the order of that access.

An assignment to a global name `x = val` is equivalent to the assignment `_ENV.x = val` (see §2.2).

The meaning of assignments to table fields and global variables (which are actually table fields, too) can be changed via metatables (see §2.4).

3.3.4 – Control Structures

The control structures **if**, **while**, and **repeat** have the usual meaning and familiar syntax:

```
stat ::= while exp do block end
stat ::= repeat block until exp
stat ::= if exp then block {elseif exp then block} [else block] end
```

Lua also has a **for** statement, in two flavors (see §3.3.5).

The condition expression of a control structure can return any value. Both **false** and **nil** test false. All values different from **nil** and **false** test true. In particular, the number 0 and the empty string also test true.

In the **repeat**–**until** loop, the inner block does not end at the **until** keyword, but only after the condition. So, the condition can refer to local variables declared inside the loop block.

The **goto** statement transfers the program control to a label. For syntactical reasons, labels in Lua are considered statements too:

```
stat ::= goto Name
stat ::= label
label ::= '::' Name '::'
```

A label is visible in the entire block where it is defined, except inside nested functions. A goto can jump to any visible label as long as it does not enter into the scope of a variable declaration. A label should not be declared where a previous label with the same name is visible, even if this other label has been declared in an enclosing block.

The **break** statement terminates the execution of a **while**, **repeat**, or **for** loop, skipping to the next statement after the loop:

```
stat ::= break
```

A **break** ends the innermost enclosing loop.

The **return** statement is used to return values from a function or a chunk (which is handled as an anonymous function). Functions can return more than one value, so the syntax for the **return** statement is

```
stat ::= return [explist] [';']
```

The **return** statement can only be written as the last statement of a block. If it is necessary to **return** in the middle of a block, then an explicit inner block can be used, as in the idiom `do return end`, because now **return** is the last statement in its (inner) block.

3.3.5 – For Statement

The **for** statement has two forms: one numerical and one generic.

The numerical **for** loop

The numerical **for** loop repeats a block of code while a control variable goes through an arithmetic progression. It has the following syntax:

```
stat ::= for Name '=' exp ',' exp [',' exp] do block end
```

The given identifier (Name) defines the control variable, which is a new read-only (`const`) variable local to the loop body (*block*).

The loop starts by evaluating once the three control expressions. Their values are called respectively the *initial value*, the *limit*, and the *step*. If the step is absent, it defaults to 1.

If both the initial value and the step are integers, the loop is done with integers; note that the limit may not be an integer. Otherwise, the three values are converted to floats and the loop is done with floats. Beware of floating-point accuracy in this case.

After that initialization, the loop body is repeated with the value of the control variable going through an arithmetic progression, starting at the initial value, with a common difference given by the step. A negative step makes a decreasing sequence; a step equal to zero raises an error. The loop continues while the value is less than or equal to the limit (greater than or equal to for a negative step). If the initial value is already greater than the limit (or less than, if the step is negative), the body is not executed.

For integer loops, the control variable never wraps around; instead, the loop ends in case of an overflow.

The generic **for** loop

The generic **for** statement works over functions, called *iterators*. On each iteration, the iterator function is called to produce a new value, stopping when this new value is **nil**. The generic **for** loop has the following syntax:

```
stat ::= for namelist in explist do block end
namelist ::= Name {‘,’ Name}
```

A **for** statement like

```
for var_1, ···, var_n in explist do body end
```

works as follows.

The names *var_i* declare loop variables local to the loop body. The first of these variables is the *control variable*, which is a read-only (`const`) variable.

The loop starts by evaluating *explist* to produce four values: an *iterator function*, a *state*, an initial value for the control variable, and a *closing value*.

Then, at each iteration, Lua calls the iterator function with two arguments: the state and the control variable. The results from this call are then assigned to the loop variables, following the rules of multiple assignments (see §3.3.3). If the control variable becomes **nil**, the loop terminates. Otherwise, the body is executed and the loop goes to the next iteration.

The closing value behaves like a to-be-closed variable (see §3.3.8), which can be used to release resources when the loop ends. Otherwise, it does not interfere with the loop.

3.3.6 – Function Calls as Statements

To allow possible side-effects, function calls can be executed as statements:

```
stat ::= functioncall
```

In this case, all returned values are thrown away. Function calls are explained in §3.4.10.

3.3.7 – Variable Declarations

Local and global variables can be declared anywhere inside a block. The declaration can include an initialization:

```
stat ::= local attnamelist ['=' explist]
stat ::= global attnamelist ['=' explist]
```

If there is no initialization, local variables are initialized with **nil**; global variables are left unchanged. Otherwise, the initialization gets the same adjustment of a multiple assignment (see §3.3.3). Moreover, for global variables, the initialization will raise a runtime error if the variable is already defined, that is, it has a non-nil value.

The list of names may be prefixed by an attribute (a name between angle brackets) and each variable name may be postfixed by an attribute:

```
attnamelist ::=  [attrib] Name [attrib] {',' Name [attrib]}
attrib ::= '<' Name '>'
```

A prefixed attribute applies to all names in the list; a postfixed attribute applies to its particular name. There are two possible attributes: `const`, which declares a *constant* or *read-only* variable, that is, a variable that cannot be used as the left-hand side of an assignment, and `close`, which declares a to-be-closed variable (see §3.3.8). Only local variables can have the `close` attribute. A list of variables can contain at most one to-be-closed variable.

Lua offers also a collective declaration for global variables:

```
stat ::= global [attrib] '*'
```

This special form implicitly declares as globals all names not explicitly declared previously. In particular, `global<const> *` implicitly declares as read-only globals all names not explicitly declared previously; see the following example:

```
global X
global<const> *
print(math.pi)   -- Ok, 'print' and 'math' are read-only
X = 1            -- Ok, declared as read-write
Y = 1            -- Error, Y is read-only
```

As noted in §2.2, all chunks start with an implicit declaration `global *`, but this preambular declaration becomes void inside the scope of any other **global** declaration. Therefore, a program that does not use global declarations or start with `global *` has free read-write access to any global; a program that starts with `global<const> *` has free read-only access to any global; and a program that starts with any other global declaration (e.g., `global none`) can only refer to declared variables.

Note that, for global variables, the effect of any declaration is only syntactical (except for the optional assignment):

```
global X <const>, _G
X = 1           -- ERROR
_ENV.X = 1      -- Ok
```

```
_G.print(X)      -- Ok
foo()            -- 'foo' can freely change any global
```

A chunk is also a block (see §3.3.2), and so variables can be declared in a chunk outside any explicit block.

The visibility rules for variable declarations are explained in §2.2.

3.3.8 – To-be-closed Variables

A to-be-closed variable behaves like a constant local variable, except that its value is *closed* whenever the variable goes out of scope, including normal block termination, exiting its block by **break**/**goto**/**return**, or exiting by an error.

Here, to *close* a value means to call its `__close` metamethod. When calling the metamethod, the value itself is passed as the first argument. If there was an error, the error object that caused the exit is passed as a second argument; otherwise, there is no second argument.

The value assigned to a to-be-closed variable must have a `__close` metamethod or be a false value. (**nil** and **false** are ignored as to-be-closed values.)

If several to-be-closed variables go out of scope at the same event, they are closed in the reverse order that they were declared.

If there is any error while running a closing method, that error is handled like an error in the regular code where the variable was defined. After an error, the other pending closing methods will still be called.

If a coroutine yields and is never resumed again, some variables may never go out of scope, and therefore they will never be closed. (These variables are the ones created inside the coroutine and in scope at the point where the coroutine yielded.) Similarly, if a coroutine ends with an error, it does not unwind its stack, so it does not close any variable. In both cases, you can either use finalizers or call `coroutine.close` to close the variables. However, if the coroutine was created through `coroutine.wrap`, then its corresponding function will close the coroutine in case of errors.

3.4 – Expressions

The basic expressions in Lua are the following:

```
exp ::= prefixexp
exp ::= nil | false | true
exp ::= Numeral
exp ::= LiteralString
exp ::= functiondef
exp ::= tableconstructor
exp ::= '...'
exp ::= exp binop exp
exp ::= unop exp
prefixexp ::= var | functioncall | '(' exp ')'
```

Numerals and literal strings are explained in §3.1; variables are explained in §3.2; function definitions are explained in §3.4.11; function calls are explained in §3.4.10; table constructors are explained in

§3.4.9. Vararg expressions, denoted by three dots ('`...`'), can only be used when directly inside a variadic function; they are explained in §3.4.11.

Binary operators comprise arithmetic operators (see §3.4.1), bitwise operators (see §3.4.2), relational operators (see §3.4.4), logical operators (see §3.4.5), and the concatenation operator (see §3.4.6). Unary operators comprise the unary minus (see §3.4.1), the unary bitwise NOT (see §3.4.2), the unary logical **not** (see §3.4.5), and the unary *length operator* (see §3.4.7).

3.4.1 – Arithmetic Operators

Lua supports the following arithmetic operators:

- `+`: addition
- `-`: subtraction
- `*`: multiplication
- `/`: float division
- `//`: floor division
- `%`: modulo
- `^`: exponentiation
- `-`: unary minus

With the exception of exponentiation and float division, the arithmetic operators work as follows: If both operands are integers, the operation is performed over integers and the result is an integer. Otherwise, if both operands are numbers, then they are converted to floats, the operation is performed following the machine's rules for floating-point arithmetic (usually the IEEE 754 standard), and the result is a float. (The string library coerces strings to numbers in arithmetic operations; see §3.4.3 for details.)

Exponentiation and float division (`/`) always convert their operands to floats and the result is always a float. Exponentiation uses the ISO C function `pow`, so that it works for non-integer exponents too.

Floor division (`//`) is a division that rounds the quotient towards minus infinity, resulting in the floor of the division of its operands.

Modulo is defined as the remainder of a division that rounds the quotient towards minus infinity (floor division).

In case of overflows in integer arithmetic, all operations *wrap around*.

3.4.2 – Bitwise Operators

Lua supports the following bitwise operators:

- `&`: bitwise AND
- `|`: bitwise OR
- `~`: bitwise exclusive OR
- `>>`: right shift
- `<<`: left shift
- `~`: unary bitwise NOT

All bitwise operations convert its operands to integers (see §3.4.3), operate on all bits of those integers, and result in an integer.

Both right and left shifts fill the vacant bits with zeros. Negative displacements shift to the other direction; displacements with absolute values equal to or higher than the number of bits in an integer result in zero (as all bits are shifted out).

3.4.3 – Coercions and Conversions

Lua provides some automatic conversions between some types and representations at run time. Bitwise operators always convert float operands to integers. Exponentiation and float division always convert integer operands to floats. All other arithmetic operations applied to mixed numbers (integers and floats) convert the integer operand to a float. The C API also converts both integers to floats and floats to integers, as needed. Moreover, string concatenation accepts numbers as arguments, besides strings.

In a conversion from integer to float, if the integer value has an exact representation as a float, that is the result. Otherwise, the conversion gets the nearest higher or the nearest lower representable value. This kind of conversion never fails.

The conversion from float to integer checks whether the float has an exact representation as an integer (that is, the float has an integral value and it is in the range of integer representation). If it does, that representation is the result. Otherwise, the conversion fails.

Several places in Lua coerce strings to numbers when necessary. In particular, the string library sets metamethods that try to coerce strings to numbers in all arithmetic operations. If the conversion fails, the library calls the metamethod of the other operand (if present) or it raises an error. Note that bitwise operators do not do this coercion.

It is always a good practice not to rely on the implicit coercions from strings to numbers, as they are not always applied; in particular, `"1"==1` is false and `"1"<1` raises an error (see §3.4.4). These coercions exist mainly for compatibility and may be removed in future versions of the language.

A string is converted to an integer or a float following its syntax and the rules of the Lua lexer. The string may have also leading and trailing whitespaces and a sign. All conversions from strings to numbers accept both a dot and the current locale mark as the radix character. (The Lua lexer, however, accepts only a dot.) If the string is not a valid numeral, the conversion fails. If necessary, the result of this first step is then converted to a specific number subtype following the previous rules for conversions between floats and integers.

The conversion from numbers to strings uses a non-specified human-readable format. To convert numbers to strings in any specific way, use the function `string.format`.

3.4.4 – Relational Operators

Lua supports the following relational operators:

- `==`: equality
- `~=`: inequality
- `<`: less than
- `>`: greater than
- `<=`: less or equal
- `>=`: greater or equal

These operators always result in **false** or **true**.

Equality (==) first compares the type of its operands. If the types are different, then the result is **false**. Otherwise, the values of the operands are compared. Strings are equal if they have the same byte content. Numbers are equal if they denote the same mathematical value.

Tables, userdata, and threads are compared by reference: two objects are considered equal only if they are the same object. Every time you create a new object (a table, a userdata, or a thread), this new object is different from any previously existing object. A function is always equal to itself. Functions with any detectable difference (different behavior, different definition) are always different. Functions created at different times but with no detectable differences may be classified as equal or not (depending on internal caching details).

You can change the way that Lua compares tables and userdata by using the `__eq` metamethod (see §2.4).

Equality comparisons do not convert strings to numbers or vice versa. Thus, `"0"==0` evaluates to **false**, and `t[0]` and `t["0"]` denote different entries in a table.

The operator `~=` is exactly the negation of equality (`==`).

The order operators work as follows. If both arguments are numbers, then they are compared according to their mathematical values, regardless of their subtypes. Otherwise, if both arguments are strings, then their values are compared according to the current locale. Otherwise, Lua tries to call the `__lt` or the `__le` metamethod (see §2.4). A comparison `a > b` is translated to `b < a` and `a >= b` is translated to `b <= a`.

Following the IEEE 754 standard, the special value NaN is considered neither less than, nor equal to, nor greater than any value, including itself.

3.4.5 – Logical Operators

The logical operators in Lua are **and**, **or**, and **not**. Like the control structures (see §3.3.4), all logical operators consider both **false** and **nil** as false and anything else as true.

The negation operator **not** always returns **false** or **true**. The conjunction operator **and** returns its first argument if this value is **false** or **nil**; otherwise, **and** returns its second argument. The disjunction operator **or** returns its first argument if this value is different from **nil** and **false**; otherwise, **or** returns its second argument. Both **and** and **or** use short-circuit evaluation; that is, the second operand is evaluated only if necessary. Here are some examples:

```
10 or 20            --> 10
10 or error()       --> 10
nil or "a"          --> "a"
nil and 10          --> nil
false and error()   --> false
false and nil       --> false
false or nil        --> nil
10 and 20           --> 20
```

3.4.6 – Concatenation

The string concatenation operator in Lua is denoted by two dots ('`..`'). If both operands are strings or numbers, then the numbers are converted to strings in a non-specified format (see §3.4.3). Otherwise, the `__concat` metamethod is called (see §2.4).

3.4.7 – The Length Operator

The length operator is denoted by the unary prefix operator #.

The length of a string is its number of bytes. (That is the usual meaning of string length when each character is one byte.)

The length operator applied on a table returns a border in that table. A *border* in a table `t` is any non-negative integer that satisfies the following condition:

```
(border == 0 or t[border] ~= nil) and
(t[border + 1] == nil or border == math.maxinteger)
```

In words, a border is any positive integer index present in the table that is followed by an absent index, plus two limit cases: zero, when index 1 is absent; and the maximum value for an integer, when that index is present. Note that keys that are not positive integers do not interfere with borders.

A table with exactly one border is called a *sequence*. For instance, the table `{10,20,30,40,50}` is a sequence, as it has only one border (5). The table `{10,20,30,nil,50}` has two borders (3 and 5), and therefore it is not a sequence. (The **nil** at index 4 is called a *hole*.) The table `{nil,20,30,nil,nil,60,nil}` has three borders (0, 3, and 6), so it is not a sequence, too. The table `{}` is a sequence with border 0.

When `t` is a sequence, `#t` returns its only border, which corresponds to the intuitive notion of the length of the sequence. When `t` is not a sequence, `#t` can return any of its borders. (The exact one depends on details of the internal representation of the table, which in turn can depend on how the table was populated and the memory addresses of its non-numeric keys.)

The computation of the length of a table has a guaranteed worst time of *O(log n)*, where *n* is the largest integer key in the table.

A program can modify the behavior of the length operator for any value but strings through the `__len` metamethod (see §2.4).

3.4.8 – Precedence

Operator precedence in Lua follows the table below, from lower to higher priority:

```
or
and
<     >     <=    >=    ~=    ==
|
~
&
<<    >>
..
+     -
*     /     //    %
unary operators (not   #     -     ~)
^
```

As usual, you can use parentheses to change the precedences of an expression. The concatenation ('`..`') and exponentiation ('^') operators are right associative. All other binary operators are left associative.

3.4.9 – Table Constructors

Table constructors are expressions that create tables. Every time a constructor is evaluated, a new table is created. A constructor can be used to create an empty table or to create a table and initialize some of its fields. The general syntax for constructors is

```
tableconstructor ::= '{' [fieldlist] '}'
fieldlist ::= field {fieldsep field} [fieldsep]
field ::= '[' exp ']' '=' exp | Name '=' exp | exp
fieldsep ::= ',' | ';'
```

Each field of the form `[exp1] = exp2` adds to the new table an entry with key `exp1` and value `exp2`. A field of the form `name = exp` is equivalent to `["name"] = exp`. Fields of the form `exp` are equivalent to `[i] = exp`, where `i` are consecutive integers starting with 1; fields in the other formats do not affect this counting. For example,

```
a = { [f(1)] = g; "x", "y"; x = 1, f(x), [30] = 23; 45 }
```

is equivalent to

```
do
  local t = {}
  t[f(1)] = g
  t[1] = "x"         -- 1st exp
  t[2] = "y"         -- 2nd exp
  t.x = 1            -- t["x"] = 1
  t[3] = f(x)        -- 3rd exp
  t[30] = 23
  t[4] = 45          -- 4th exp
  a = t
end
```

The order of the assignments in a constructor is undefined. (This order would be relevant only when there are repeated keys.)

If the last field in the list has the form `exp` and the expression is a multires expression, then all values returned by this expression enter the list consecutively (see §3.4.12).

The field list can have an optional trailing separator, as a convenience for machine-generated code.

3.4.10 – Function Calls

A function call in Lua has the following syntax:

```
functioncall ::= prefixexp args
```

In a function call, first prefixexp and args are evaluated. If the value of prefixexp has type *function*, then this function is called with the given arguments. Otherwise, if present, the prefixexp `__call` metamethod is called: its first argument is the value of prefixexp, followed by the original call arguments (see §2.4).

The form

```
functioncall ::= prefixexp ':' Name args
```

can be used to emulate methods. A call `v:name(args)` is syntactic sugar for `v.name(v, args)`, except that v is evaluated only once.

Arguments have the following syntax:

```
args ::= ‘(’ [explist] ‘)’
args ::= tableconstructor
args ::= LiteralString
```

All argument expressions are evaluated before the call. A call of the form `f{fields}` is syntactic sugar for `f({fields})`; that is, the argument list is a single new table. A call of the form `f'string'` (or `f"string"` or `f[[string]]`) is syntactic sugar for `f('string')`; that is, the argument list is a single literal string.

A call of the form `return functioncall` not in the scope of a to-be-closed variable is called a *tail call*. Lua implements *proper tail calls* (or *proper tail recursion*): In a tail call, the called function reuses the stack entry of the calling function. Therefore, there is no limit on the number of nested tail calls that a program can execute. However, a tail call erases any debug information about the calling function. Note that a tail call only happens with a particular syntax, where the **return** has one single function call as argument, and it is outside the scope of any to-be-closed variable. This syntax makes the calling function return exactly the returns of the called function, without any intervening action. So, none of the following examples are tail calls:

```
return (f(x))        -- results adjusted to 1
return 2 * f(x)      -- result multiplied by 2
return x, f(x)       -- additional results
f(x); return         -- results discarded
return x or f(x)     -- results adjusted to 1
```

3.4.11 – Function Definitions

The syntax for a function definition is

```
functiondef ::= function funcbody
funcbody ::= ‘(’ [parlist] ‘)’ block end
```

The following syntactic sugar simplifies function definitions:

```
stat ::= function funcname funcbody
stat ::= local function Name funcbody
stat ::= global function Name funcbody
funcname ::= Name {‘.’ Name} [‘:’ Name]
```

The statement

```
function f () body end
```

translates to

```
f = function () body end
```

The statement

```
function t.a.b.c.f () body end
```

translates to

```
t.a.b.c.f = function () body end
```

The statement

```
local function f () body end
```

translates to

```
local f; f = function () body end
```

not to

```
local f = function () body end
```

(This only makes a difference when the body of the function contains recursive references to `f`.) Similarly, the statement

```
global function f () body end
```

translates to

```
global f; global f = function () body end
```

The second **global** makes the assignment an initialization, which will raise an error if that global is already defined.

The *colon* syntax is used to emulate *methods*, adding an implicit extra parameter `self` to the function. Thus, the statement

```
function t.a.b.c:f (params) body end
```

is syntactic sugar for

```
t.a.b.c.f = function (self, params) body end
```

A function definition is an executable expression, whose value has type *function*. When Lua precompiles a chunk, all its function bodies are precompiled too, but they are not created yet. Then, whenever Lua executes the function definition, the function is *instantiated* (or *closed*). This function instance, or *closure*, is the final value of the expression.

Results are returned using the **return** statement (see §3.3.4). If control reaches the end of a function without encountering a **return** statement, then the function returns with no results.

There is a system-dependent limit on the number of values that a function may return. This limit is guaranteed to be at least 1000.

Parameters

Parameters act as local variables that are initialized with the argument values:

```
parlist ::= namelist [‘,’ varargparam] | varargparam
varargparam ::= ‘...’ [Name]
```

When a Lua function is called, it adjusts its list of arguments to the length of its list of parameters (see §3.4.12), unless the function is a *variadic function*, which is indicated by three dots ('`...`') at the end of its parameter list. A variadic function does not adjust its argument list; instead, it collects all extra arguments and supplies them to the function through a *vararg table*. In that table, the values at indices 1, 2, etc. are the extra arguments, and the value at index "`n`" is the number of extra arguments.

As an example, consider the following definitions:

```
function f(a, b) end
function g(a, b, ...) end
function r() return 1,2,3 end
```

Then, we have the following mapping from arguments to parameters and to the vararg table:

```
CALL             PARAMETERS

f(3)             a=3, b=nil
f(3, 4)          a=3, b=4
f(3, 4, 5)       a=3, b=4
f(r(), 10)       a=1, b=10
f(r())           a=1, b=2

g(3)             a=3, b=nil, va. table ->  {n = 0}
g(3, 4)          a=3, b=4,   va. table ->  {n = 0}
g(3, 4, 5, 8)    a=3, b=4,   va. table ->  {5, 8, n = 2}
g(5, r())        a=5, b=1,   va. table ->  {2, 3, n = 2}
```

A vararg table in a variadic function can have an optional name, given after the three dots. When present, that name denotes a read-only local variable that refers to the vararg table. If the vararg table does not have a name, it can only be accessed through a vararg expression.

A vararg expression is also written as three dots, and its value is a list of the values in the vararg table, from 1 to the integer value at index "`n`". (Therefore, if the code does not modify the vararg table, this list corresponds to the extra arguments in the function call.) This list behaves like the results from a function with multiple results (see §3.4.12).

As an optimization, if the vararg table satisfies some conditions, the code does not create an actual table and instead translates the indexing expressions and the vararg expressions into accesses to the internal vararg data. The conditions are as follows: If the vararg table has a name, that name is not an upvalue in a nested function and it is used only as the base table in the syntactic constructions `t[exp]` or `t.id`. Note that an anonymous vararg table always satisfy these conditions.

3.4.12 – Lists of Expressions, Multiple Results, and Adjustment

Both function calls and vararg expressions can result in multiple values. These expressions are called *multires expressions*.

When a multires expression is used as the last element of a list of expressions, all results from the expression are added to the list of values produced by the list of expressions. Note that a single expression in a place that expects a list of expressions is the last expression in that (singleton) list.

These are the places where Lua expects a list of expressions:

- A **return** statement, for instance `return e1,e2,e3` (see §3.3.4).
- A table constructor, for instance `{e1,e2,e3}` (see §3.4.9).
- The arguments of a function call, for instance `foo(e1,e2,e3)` (see §3.4.10).
- A multiple assignment, for instance `a,b,c = e1,e2,e3` (see §3.3.3).
- A local or global declaration, which is similar to a multiple assignment.
- The initial values in a generic **for** loop, for instance `for k in e1,e2,e3 do ... end` (see §3.3.5).

In the last four cases, the list of values from the list of expressions must be *adjusted* to a specific length: the number of parameters in a call to a non-variadic function (see §3.4.11), the number of variables in a multiple assignment or a declaration, and exactly four values for a generic **for** loop. The *adjustment* follows these rules: If there are more values than needed, the extra values are thrown away; if there are fewer values than needed, the list is extended with **nil**'s. When the list of expressions ends with a multires expression, all results from that expression enter the list of values before the adjustment.

When a multires expression is used in a list of expressions without being the last element, or in a place where the syntax expects a single expression, Lua adjusts the result list of that expression to one element. As a particular case, the syntax expects a single expression inside a parenthesized expression; therefore, adding parentheses around a multires expression forces it to produce exactly one result.

We seldom need to use a vararg expression in a place where the syntax expects a single expression. (Usually it is simpler to add a regular parameter before the variadic part and use that parameter.) When there is such a need, we recommend assigning the vararg expression to a single variable and using that variable in its place.

Here are some examples of uses of multires expressions. In all cases, when the construction needs "the n-th result" and there is no such result, it uses a **nil**.

```
print(x, f())      -- prints x and all results from f().
print(x, (f()))    -- prints x and the first result from f().
print(f(), x)      -- prints the first result from f() and x.
print(1 + f())     -- prints 1 added to the first result from f().
local x = ...      -- x gets the first vararg argument.
x,y = ...          -- x gets the first vararg argument,
                   -- y gets the second vararg argument.
x,y,z = w, f()     -- x gets w, y gets the first result from f(),
                   -- z gets the second result from f().
x,y,z = f()        -- x gets the first result from f(),
                   -- y gets the second result from f(),
                   -- z gets the third result from f().
x,y,z = f(), g()   -- x gets the first result from f(),
                   -- y gets the first result from g(),
                   -- z gets the second result from g().
x,y,z = (f())      -- x gets the first result from f(), y and z get nil.
return f()         -- returns all results from f().
return x, ...      -- returns x and all received vararg arguments.
return x,y,f()     -- returns x, y, and all results from f().
{f()}              -- creates a list with all results from f().
{...}              -- creates a list with all vararg arguments.
{f(), 5}           -- creates a list with the first result from f() and 5.
```